

# Using Multi-Agent Systems For Predicting and Preventing Natural Disasters

**Abstract.** A warning that constantly flips on and off is not a warning people can trust. Early warning systems are an end-to-end capability intended to enable timely, actionable decisions that reduce disaster impacts, but practical deployments must operate under noisy, incomplete, and intermittently missing observations [1,2]. This paper proposes FloodMAS, a multi-agent flood early-warning prototype that demonstrates how agent decomposition plus calibrated machine learning can produce operationally stable alarms rather than brittle, flapping triggers. FloodMAS simulates multi-zone, IoT-like sensing where sensor agents emit rainfall and water-level readings with noise/dropout; zone “edge” agents compute rolling-window features and estimate short-horizon flood risk (flood defined consistently as zone mean water level exceeding a flood threshold within the next  $T$  discrete steps, a sped-up experimental abstraction); and a coordinator agent fuses zone evidence into a stable global alarm. The ML risk estimator is trained using standard ensembles and post-hoc probability calibration so that its outputs are meaningful for thresholding and policy logic [3,4]. Stability is enforced via explicit guardrails (hysteresis, debouncing, consensus gating, and health-aware degradation), reflecting established alarm-management practice that uses deadband and delays to mitigate nuisance alarms [5]. Empirically, FloodMAS achieves near-perfect offline discrimination (ROC-AUC 0.998; F1 0.989; Brier 0.0097) and, in extreme scenarios, substantially higher ALERT-only F1 than a minimal threshold baseline while maintaining fewer global alarm toggles (greater stability).

**Keywords:** Multi-agent system, ML, AI, Python, Natural Disasters, Model Training, Random Forest

## 1 Introduction

Natural disasters are one of the most common contributors of unpredictable damages and hence, early warning systems are widely recognized as a foundational capability for reducing loss of life and damage from weather - and climate-related hazards, including floods [1]. In the multi-hazard framing adopted by the United Nations system, an early warning system is an integrated pipeline spanning monitoring and forecasting, risk assessment, communication, and preparedness, all designed to enable timely action before hazardous events [2]. In practice, operational deployments must function under conditions that are hostile to brittle automation: measurements arrive from heterogeneous sensors, often across remote areas and organizational boundaries; communication links can be unreliable; and warning delivery itself can be disrupted by the hazard [6]. These realities motivate system designs that treat missing data, transient noise, and partial failures as normal operating conditions rather than exceptional cases.

A common first approach to automated detection is a fixed threshold heuristic (for example, “trigger when a signal crosses a limit”). Even though it is simple, such rules are prone to false alarms and alarm “flapping” when readings hover near decision boundaries or when short-lived disturbances produce transient spikes. In real warning workflows, unstable alerting is not merely an accuracy issue because it also lowers trust, increases operator burden, and complicates downstream action since we cannot always react to false alarms. What is needed is not only accurate detection, but also operational stability in the sense of a mechanism for turning noisy, incomplete signals into decisions that change state only when there is sustained evidence.

This paper presents the system that we will call FloodMAS, a multi-agent flood early-warning system that combines a calibrated machine-learning (ML) risk estimator and explicit guardrails that enforce stable alarm dynamics. Multi-agent architectures are a natural fit for distributed sensing because they separate concerns such as sensor management, local processing, communication, and adaptation to changing device availability and that improves modularity and responsiveness in dynamic environments [7]. FloodMAS follows a sensing, aggregation and then coordination pattern: sensor agents emit local readings, edge aggregator agents compute rolling temporal features and produce a zone-level risk score, and a coordinator agent fuses zone-level evidence into a global alarm. The ML layer outputs a probability-like risk estimate that is intended to be interpretable as likelihood, motivating the use of probability calibration methods (like isotonic regression) to better align predicted confidence with empirical correctness [8,9]. The guardrails layer then transforms these risk estimates into stable discrete states using mechanisms such as hysteresis, debouncing, consensus gating across zones, and health-aware degradation when sensing quality deteriorates.

FloodMAS is evaluated in a controlled multi-zone simulator designed for reproducible stress testing under scenario variations (for example typical vs. extreme precipitation, wet vs. dry initial soil, and sensor dropout/noise). Time is represented as discrete fictional “steps” and they represent an abstracted time frame so that we can get a more rapid examination and experimentation while also still keeping the structure of a real system. As a showcase, we are comparing against a simple threshold baseline that uses the same input streams and minimal stabilizers (small hysteresis margin and short debouncing), but no multi-agent coordination, no health-awareness, and no learned risk model that the new system provides.

The contributions of this work are:

- A multi-agent early-warning architecture for disaster-style monitoring that cleanly separates sensing, local inference, and global coordination [7].
- A calibrated ML + guardrails design pattern for producing decisions that are both predictive and operationally stable, grounded in established probability calibration methodology [8,9].
- A scenario-based experimental methodology that tests robustness to noise, missing data, and extreme conditions, aligned with the communication and reliability challenges documented for real early warning systems [6],

Although the main topic of this paper are floods, we suggest that such system can be applied to a variety of other natural disasters such as earthquakes, tornadoes and so on and so forth.

## 2 Technology and Problem Overview

**Floods-** In operational terms, flooding is an overflow of water onto normally dry land, including inundation driven by rising water in waterways or local ponding after rainfall [10]. A hydrologic flood in rivers is commonly described as high streamflow overtopping natural or artificial banks [11]. Floods can develop over longer periods, while flash floods are characterized by very rapid onset after intense rainfall (often within hours)[12,13]. In our system we treat early warning as a short-horizon prediction and stabilization problem, given recent rainfall and water-level dynamics, and estimate whether conditions are likely to cross a flood threshold within the next  $X$  time steps.

**Real physical signals and “flood-relevant” parameters-** While real flood forecasting can require detailed hydrology and hydraulic modeling, many early-warning pipelines still revolve around a small set of directly measurable drivers like rainfall accumulation/intensity and the resulting rise in water level or stage. Because of that we center around rainfall as the forcing input and zone water level as a signal for local risk, with “flood” defined consistently as zone mean water level exceeding a fixed threshold within the horizon  $T$ . This mirrors common operational language where flooding corresponds to water exceeding normal containment (banks, channels, drainage capacity)[10,11].

**Rolling-window features in Python -** A key step in early warning is to transform raw sensor streams into short-term summaries that capture sustained conditions. We use rolling statistics such as moving averages (like recent mean water level) and windowed accumulations. Rolling-window computation is a standard time-series primitive supported directly in pandas via `DataFrame.rolling(...)` and related window operators [14]. These things provide the model and the guardrails with temporal context and reduce sensitivity to single-step noise.

**Lightweight ML models for risk estimation (Random Forest and Gradient Boosting)-** FloodMAS uses standard, widely adopted scikit-learn classifiers as the zone-level risk estimator. A Random Forest is an ensemble that fits many decision trees on different subsamples and averages their predictions to improve accuracy and control overfitting[15]. A Gradient Boosting Classifier builds an additive model in stages, fitting trees to improve performance under a differentiable loss such as log loss (a natural choice for probabilistic classification outputs like this one)[15]. In FloodMAS, these models serve as local (per-zone) probability-like predictors that can be thresholded and combined downstream by the multi-agent coordination logic.

**Probability calibration and probabilistic evaluation -** Because operational decisions often threshold predicted risk, it is valuable for model scores to behave like calibrated probabilities rather than arbitrary confidence values. FloodMAS therefore applies probability calibration (notably non-parametric isotonic calibration) following established methodology for transforming classifier scores into better probability

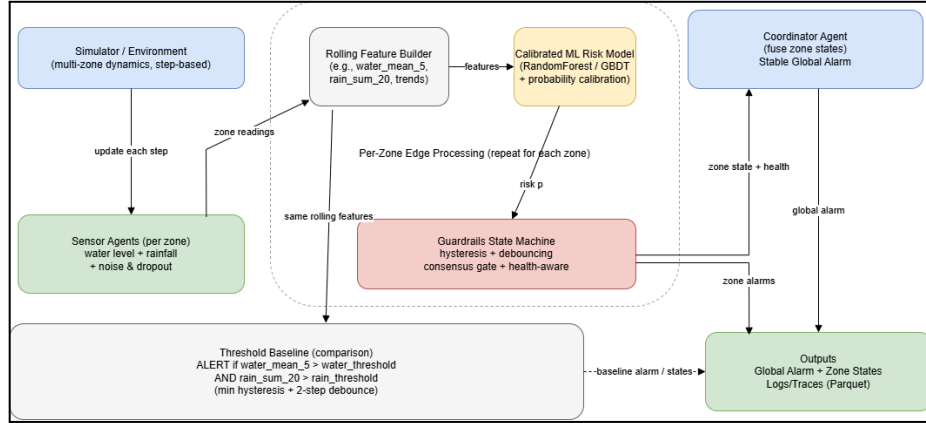
estimates[8]. In reality calibration is supported in scikit-learn via the calibration framework (like CalibratedClassifierCV) and is commonly assessed using tools such as calibration curves and proper scoring rules (like Brier loss)[4,16].

Data and experiment implementation - FloodMAS is implemented in Python using standard scientific computing tooling (NumPy/pandas for feature construction and analysis, scikit-learn for model training, calibration, and evaluation). Scenario runs and step-by-step traces are stored in Apache Parquet, an open, column-oriented file format designed for efficient storage and retrieval of analytic datasets[17]. This supports reproducible experiments, rapid plotting of timelines (risk vs. alarm state over steps), and clean comparisons between the MAS pipeline and the threshold baseline.

### 3 System and Implementation Overview (FloodMAS)

#### 3.1 Multi-Agent Architecture and Responsibilities

FloodMAS is organized as a hierarchical multi-agent pipeline that separates sensing, local inference, and global coordination. This decomposition follows a common MAS principle in sensor-network settings where treat sensors as devices managed by higher-layer agents that handle processing, communication, and adaptation to changing device availability[7].



**Fig 1.** Architecture of the Multi-Agent System

The system consists of Sensor agents (per zone)- Produce local observations (rainfall and water level proxies) with realistic imperfections such as noise and dropout so that the system does not end up being too “perfect”.

Edge aggregation agents- Maintain short rolling buffers, compute windowed features, invoke the calibrated ML model to obtain a continuous risk score  $p \in [0,1]$ , and apply a guardrails state machine to produce a discrete zone state.

Coordinator agent (global for everyone)- Fuses zone-level outputs into a stable global alarm, designed to avoid unnecessary state oscillation and to reflect multi-zone evidence rather than isolated spikes.

Optional mitigation agents- A hook for actuation policies (like pumps/gates) that react

to sustained alerts so that they somewhat react to the sensors. They are kept optional to ensure the paper’s focus remains on warning stability and decision logic.

### 3.2 Per-Step Data Flow (Discrete time “Steps”)

Time is modeled as discrete steps (a sped-up experimental abstraction). At each step, FloodMAS executes the following pipeline:

1. Environment update: The simulation advances the underlying zone dynamics (rainfall forcing and water-level evolution are handled by the simulator which uses simple ways to cause floods).
2. Sensing: Sensor agents sample local signals and emit readings; missing data may occur due to dropout.
3. Zone aggregation and inference: Each zone edge agent
  - filters and imputes missing values to maintain continuity,
  - computes rolling-window features (moving averages/sums/trends),
  - runs calibrated ML inference to obtain a risk estimate  $p$ ,
  - passes  $p$  into the guardrails state machine to produce a discrete zone state.
4. Coordination: The coordinator combines zone states into a global alarm, using conservative fusion (for example, if an alarm in any zone is in ALERT, with global risk tracked as an aggregate such as max zone risk).
5. Logging: The system logs per-step traces for analysis and figure generation; results are stored in columnar Parquet for efficient time-series retrieval and plotting[17].

### 3.3 Guardrails: Stable Discrete Decisions from Noisy Risk

The guardrails layer is the operational core: it converts continuous risk  $pp$  into a stable discrete state while accounting for sensor unreliability. This is aligned with established alarm-system practice, where deadband/hysteresis and time filtering (debouncing) are used specifically to prevent alarm “chattering” when a variable oscillates near a setpoint[18].

FloodMAS uses a small state machine per zone where it can go something like NORMAL  $\rightarrow$  SUSPECTED  $\rightarrow$  ALERT  $\rightarrow$  COOLDOWN  $\rightarrow$  NORMAL, governed by four mechanisms:

1. **Hysteresis (deadband):**

Two thresholds are used: an escalation threshold and a de-escalation where threshold down  $<$  threshold up. This gap prevents rapid toggling when  $p$  hovers near a single boundary, matching the deadband concept used to prevent chattering alarms[18].

2. **Debouncing (time filtering):**

A state change requires  $K$  consecutive steps meeting the condition. Time filtering is a standard anti-chatter mechanism in alarm practice[18].

3. **Consensus gating:**

Escalation to ALERT additionally requires agreement among operational sensors in the zone. This is motivated by a fault-tolerance idea that when multiple redundant sources exist, a **majority voter** can mask isolated faults by relying on agreement among replicas[19]. In FloodMAS, “replicas” correspond to multiple sensors observing the same zone phenomenon.

4. **Health-aware degradation:**

Each zone tracks a simple health indicator (fraction of sensors currently operational). When health drops below a minimum, guardrails become more conservative to reduce false alarms when evidence quality is poor. This follows the same philosophy as alarm-system guidance: when oscillations or noisy behavior are expected (vibrations, turbulence and similar), additional filtering and suppression mechanisms are justified to avoid nuisance alarms[18].

### 3.4 Ground Truth and Label Interface

To keep the evaluation interpretable, FloodMAS uses a consistent operational definition of ground truth for supervised learning and for plotting: a zone is treated as flooded when its zone-mean water level exceeds a fixed flood threshold, and the prediction target is whether the zone will exceed that threshold within the next  $T$  steps (short-horizon early warning). The key point is not hydrologic realism, but a repeatable contract that links sensor dynamics  $\rightarrow$  features  $\rightarrow$  risk  $\rightarrow$  stable alarms.

### 3.5 Implementation Notes and Outputs

FloodMAS is implemented in Python with standard scientific tooling. A central design choice is to produce step-aligned traces and store them in Parquet to support reproducible analysis and paper-ready figures (timelines, stability/flapping, robustness under dropout)[17].

## 4 Results Summary

### 4.1 Offline ML Model Quality (Held-Out Test Set)

FloodMAS trains a zone-level classifier to predict whether a zone will exceed the flood threshold within the next  $T$  steps. Table I summarizes held-out test performance using standard classification metrics: ROC-AUC measures discrimination across thresholds[20], F1 is the harmonic mean of precision and recall[21], and the Brier score measures mean-squared error between predicted probabilities and outcomes (lower is better)[22].

| Metric            | Value             |
|-------------------|-------------------|
| ROC-AUC           | 0.998             |
| F1                | 0.989             |
| Precision         | 0.992             |
| Recall            | 0.986             |
| Accuracy          | 0.988             |
| Brier score       | 0.0097            |
| 5-fold CV ROC-AUC | 0.998 $\pm$ 0.001 |

**Fig 2.** Calibrated ML model performance (test set: 64,800 samples)

The confusion matrix is: TN = 30,000; FP = 291; FN = 500; TP = 34,009 (total 64,800). This corresponds to a false-positive rate of  $291/(30,000+291)=0.96\%$  and a miss rate of  $500/(34,009+500)=1.45\%$ . The low Brier score indicates that the model's probability outputs are well-behaved for thresholding and downstream policy logic, consistent with the purpose of probability calibration methods in scikit-learn[4].

#### 4.2 Scenario-Level Operational Performance (MAS vs. Baseline)

We evaluate FloodMAS end-to-end on four scenario “hero runs,” each with 4 zones over 300 steps (1,200 zone-steps). FloodMAS uses guardrails thresholds  $THUP=0.6TH$ ,  $THDOWN=0.4$  with debouncing  $KUP=3$ ,  $KDOWN=5$ , consensus minimum  $CONS\_MIN=0.5$ , and health degradation threshold  $HEALTH\_MIN=0.6$ . The baseline is a minimal heuristic where ALERT if  $water\_mean\_5 > water\_threshold$  and  $rain\_sum\_20 > rain\_threshold$ , with only a small hysteresis margin and 2-step debouncing (no consensus, no health awareness, no ML).

| Scenario       | MAS Precision / Recall / F1 | Baseline Precision / Recall / F1 | MAS First Alarm | Baseline First Alarm | First Flood | MAS Delay | Baseline Delay | MAS Global Toggles | Baseline Global Toggles |
|----------------|-----------------------------|----------------------------------|-----------------|----------------------|-------------|-----------|----------------|--------------------|-------------------------|
| extreme_wet    | 1.000 / 0.531 / 0.694       | 1.000 / 0.127 / 0                | 52              | 74                   | 1           | 51        | 73             | 1                  | 2                       |
| extreme_dropou | 1.000 / 0.513 / 0.678       | 1.000 / 0.127 / 0                | 57              | 74                   | 1           | 56        | 73             | 1                  | 2                       |
| extreme_dry    | 1.000 / 0.490 / 0.658       | 1.000 / 0.127 / 0                | 62              | 74                   | 1           | 61        | 73             | 1                  | 2                       |

**Fig 3.** Hero scenario comparison (ALERT-only detection). Detection delay is reported in steps

In the three extreme scenarios, both systems achieve precision = 1.0 (no false positives), but FloodMAS achieves materially higher recall and F1. This gap reflects the core design intent: FloodMAS can sustain alerts under extreme conditions using calibrated risk plus guardrails, whereas the baseline reacts later and covers fewer flooded zone-steps. The simulator is designed to test system behavior under controlled conditions, not to maximize hydrologic realism hence why both of the systems performed decently, still, the MAS system still showed noticeable advantages over the baseline one!

Still, On the hero runs it looks good because precision = 1.0, but precision alone is misleading here because from the given table, the baseline is late and misses most of the flooded periods, in extreme scenarios baseline recall is 0.127, while FloodMAS recall is about 0.49-0.53, baseline first alarm is ~74, while FloodMAS is ~52-62 (earlier by ~12-22 steps). These numbers may look close but in reality they present **serious differences** in how the story plays out.

#### 4.3 Stability Under Noise, Spikes, and Dropout

In normal\_wet, no flooding occurs and both systems correctly remain silent. Notably, the ML risk signal still exhibits transient excursions: 71 individual zone-steps exceed the escalation threshold THUP, yet FloodMAS produces zero false alarms. This demonstrates why the guardrails layer is central, hysteresis and multi-step debouncing convert “momentary high risk” into “actionable alert only under sustained evidence,” reducing the chance of alarm flapping, which as we mentioned at the start, could make users lose trust in the system.

In extreme\_dropout\_50, FloodMAS maintains near-identical global stability (1 global toggle, same as no-dropout extreme\_wet) while detection delay increases by only 5 steps relative to extreme\_wet. The baseline shows the same toggle count pattern as extreme\_wet (2 toggles), indicating greater global flapping even without any explicit coordination or health-aware logic.

#### 4.4 Warm-Up Effects and Step-Based Lead Time

In scenarios where flooding begins at step 1, a non-zero delay is expected because rolling-window features require short buffer warm-up (like 5–20 steps), and guardrails intentionally require multiple consecutive confirmations before escalating. In continuous monitoring deployments, this delay primarily corresponds to “boot/warm-up” behavior rather than persistent latency during steady-state operation.

## 5 Related Work

Flood forecasting is commonly addressed through physically based hydrologic/hydrodynamic models, which can provide detailed estimates but often face compute constraints when high resolution and rapid turnaround are required. Recent work shows how HPC and multi-GPU acceleration can make large-scale, high-resolution flood simulations feasible within operational timeframes, highlighting the continued importance of physics-based modeling for early warning contexts [23]. FloodMAS is complementary to this direction, it is not proposed as a hydrodynamic



solver, but as an operational decision pipeline emphasizing robust alarm stability under noisy and missing sensor-like inputs.

A substantial literature studies data-driven flood prediction, spanning classical ML and deep learning, and reviews describe ML as a practical alternative or complement when explicit physical modeling is costly or when rapid inference is required [24]. Recent examples include real-time flood forecasting using data-driven models such as neural networks and gradient-boosted regressors under event-focused data regimes [25]. FloodMAS follows this line by using supervised learning on rolling-window features, but places the main emphasis on how risk estimates are transformed into stable operational alerts (rather than maximizing hydrologic fidelity).

Multi-agent architectures are frequently motivated by distributed, dynamic environments where responsibilities such as sensing, local processing, coordination, and adaptation can be separated into interacting components. For sensor networks, a representative architecture explicitly treats sensors as devices managed by higher-layer agents, arguing for separation of concerns and modular integration [7]. In disaster response and emergency management, MAS has been explored for coordination, task allocation, and decision support, including agent-based coordination technologies in emergency response contexts [26] and human-agent teaming systems aimed at aiding responders with decentralized coordination and decision support [27]. More recent “agentic AI” work proposes orchestrated multi-agent frameworks for integrating heterogeneous tools and data streams into cohesive disaster-management decision pipelines [28]. FloodMAS aligns with this MAS motivation, using agents primarily to structure a robust early-warning workflow (sensor  $\rightarrow$  edge inference  $\rightarrow$  coordinator), with explicit stability guardrails as first-class system behavior.

Many classifiers produce scores that are not well-calibrated probabilities, which can be problematic when downstream policies threshold risk or compare confidence across conditions. Post-hoc calibration methods such as Platt scaling and isotonic regression are established approaches for improving probability quality; empirical studies show calibration can substantially improve probabilistic reliability depending on the model family and calibration-set size [9]. FloodMAS adopts calibration specifically to make the risk signal more meaningful for guardrails and thresholding, i.e., to reduce the mismatch between “high score” and “high likelihood.”

In industrial alarm management, standards and training materials explicitly discuss mechanisms like deadband/hysteresis and on-delay/off-delay (time filtering) as practical tools to prevent nuisance alarms and chattering. An ISA overview of alarm management standards situates alarm handling as an engineering lifecycle problem [29], while instructional material for ISA 18.2 describes deadband (hysteresis) and delay attributes as common anti-chatter techniques [30]. FloodMAS applies these ideas in a disaster-monitoring context: rather than emitting a global alarm directly from raw signals, it enforces stability through hysteresis, debouncing, and multi-zone gating so that state changes reflect sustained evidence rather than transient excursions.

Positioning. Across these threads—flood forecasting (physics-based and ML), multi-agent disaster systems, probability calibration, and alarm management - FloodMAS is positioned as an integrated early-warning design pattern: distributed agents generate and aggregate evidence, calibrated ML produces interpretable

short-horizon risk, and explicit guardrails convert risk into stable zone/global alarms suitable for operational use.

## 6 Conclusion and Future Work

FloodMAS demonstrates a multi-agent approach to flood early warning that prioritizes stable decisions under noisy and partially missing sensor streams. The system decomposes the pipeline into sensor agents, zone/edge agents (rolling features + calibrated ML risk estimation), and a coordinator agent (global fusion), and then enforces stability via explicit guardrails (hysteresis, debouncing, consensus gating, and health-aware behavior). This aligns with the end-to-end framing of early warning systems as an integrated capability spanning detection/monitoring, warning communication, and preparedness/response[1], while focusing the technical contribution on the decision layer that converts uncertain evidence into reliable alert states. In scenario-based evaluations, FloodMAS produces stronger alerting performance than a minimal threshold baseline and avoids unnecessary alarm “flapping,” illustrating how calibration and guardrails can turn continuous risk into actionable, stable warnings suitable for disaster-style monitoring.

Future work can expand the system into many different directions. First of all, use of real-data validation and time mapping to evaluate with real hydrometeorological/sensor datasets and map discrete “steps” to physical time while retaining the same ground-truth contract and rolling-feature interface[1]. Also as mentioned before, this system could be extended to topics beyond floods (e.g., heat, wildfire, landslides) in line with multi-hazard early warning guidance emphasizing coordinated, compatible mechanisms across hazards[2]. Additionally, adding drift detection and periodic recalibration to preserve probabilistic reliability as regimes change, building on established calibration methodology for probabilistic classifiers could be an interesting move[8]. At the end, Deployment-grade coordination where we study distributed execution under network delay and partial failures (edge compute constraints, message loss, zone partitioning), and integrate operator-facing explanations and escalation policies so that warnings remain interpretable and actionable within the broader early-warning workflow[1].

## References

1. World Meteorological Organization: Early Warning Systems. <https://wmo.int/topics/early-warning-system>. Last accessed 2025/05/20
2. UNDRR: Multi-hazard Early Warning Systems: A Checklist. In: Global Platform for Disaster Risk Reduction (2022). <https://globalplatform.undrr.org/2022/sites/default/files/2022-02/Multi-hazard%20Early%20Warning%20Systems-%20A%20Checklist.pdf>. Last accessed 2025/11/20.
3. Pedregosa, F., et al.: Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12, pp. 2825–2830 (2011).
4. Scikit-learn Developers: Probability Calibration. *Scikit-learn Documentation*. <https://scikit-learn.org/stable/modules/calibration.html>. Last accessed 2026/01/20.

5. International Society of Automation: ANSI/ISA-18.2-2016 - Management of Alarm Systems for the Process Industries. Research Triangle Park, NC (2016).
6. Leonard, M., et al.: A framework for understanding and generating multi-hazard contexts for infrastructure. *Natural Hazards and Earth System Sciences* 10(11), 2215–2227 (2010). <https://doi.org/10.5194/nhess-10-2215-2010>.
7. Mousa, M., Zhang, X., Claudel, C.: Flash Flood Warning System via a Wireless Sensor Network and Distributed Control. *Sensors* 9(12), 10244–10260 (2009). <https://doi.org/10.3390/s91210244>.
8. Zadrozny, B., Elkan, C.: Transforming classifier scores into accurate multiclass probability estimates. In: *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 608–613 (2002).
9. Niculescu-Mizil, A., Caruana, R.: Predicting good probabilities with supervised learning. In: *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pp. 625–632 (2005).
10. National Weather Service: Flood and Flash Flood Definitions. [https://www.weather.gov/mrx/flood\\_and\\_flash](https://www.weather.gov/mrx/flood_and_flash). Last accessed 2026/01/11.
11. U.S. Geological Survey (USGS): Floods: Recurrence intervals and 100-year floods. <https://www.usgs.gov/centers/new-jersey-water-science-center/floods-recurrence-intervals-and-100-year-floods>. Last accessed 2026/01/10.
12. National Weather Service: Flash Flood Definition. <https://www.weather.gov/phi/FlashFloodingDefinition>. Last accessed 2026/02/10.
13. National Research Council: *Completing the Forecast: Characterizing and Communicating Uncertainty for Better Decisions Using Weather and Climate Forecasts*. The National Academies Press, Washington, DC (2006). <https://doi.org/10.17226/11128>.
14. The Pandas Development Team: `pandas.DataFrame.rolling` — pandas documentation. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.rolling.html>. Last accessed 2025/12/20.
15. Scikit-learn Developers: `sklearn.ensemble.RandomForestClassifier` — pandas documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>. Last accessed 2024/05/20.
16. Scikit-learn Developers: `sklearn.calibration.CalibratedClassifierCV` — scikit-learn documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.calibration.CalibratedClassifierCV.html>. Last accessed 2024/05/20.
17. Apache Software Foundation: Apache Parquet. <https://parquet.apache.org/>. Last accessed 2024/05/20.
18. Electric Power Research Institute (EPRI): Elements of Effective Alarm Management and Display Design. Product ID 1010076 (2005). <https://restservice.epri.com/publicdownload/000000000001010076/0/Product>.
19. Mossé, D.: Fault Tolerance. Course Materials, University of Pittsburgh. <https://people.cs.pitt.edu/~melhem/courses/3410p/ft.pdf>. Last accessed 2026/02/10.
20. Scikit-learn Developers: `sklearn.metrics.roc_auc_score` documentation. [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc\\_auc\\_score.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html). Accessed 2026-02-3.
21. Scikit-learn Developers: `sklearn.metrics.f1_score` documentation. [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1\\_score.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html). Accessed 2026-02-10.
22. Scikit-learn Developers: `sklearn.metrics.brier_score_loss` documentation. [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.brier\\_score\\_loss.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.brier_score_loss.html). Accessed 2026-01-3.

23. Author(s): Title of the article. *Natural Hazards and Earth System Sciences* 26, 85–102 (2026). <https://doi.org/10.5194/nhess-26-85-2026>.
24. Khalaf, M., et al.: A Low-Cost Wireless Sensor Network for Early Flood Detection and Monitoring. *Water* 10(11), 1536 (2018). <https://doi.org/10.3390/w10111536>. Accessed 2026-01-13.
25. Author(s): Title of the preprint. *EGUsphere* [Preprint] (2024). <https://doi.org/10.5194/egusphere-2023-2621>. Accessed 2026-02-10.
26. Mousa, M., Zhang, X., Claudel, C.: A Wireless Sensor Network for Real-time Flash Flood Detection in Deserts. In: *Proc. of the 7th Int. Conf. on Autonomous Agents and Multiagent Systems (AAMAS 2008) – Volume 3*, pp. 1655–1656 (2008). [https://www.ifaamas.org/Proceedings/aamas2008/proceedings/pdf/demo/AAMAS08\\_demo27.pdf](https://www.ifaamas.org/Proceedings/aamas2008/proceedings/pdf/demo/AAMAS08_demo27.pdf). Accessed 2026-01-20.
27. Feng, W., Zilberstein, S., Meuleau, N.: Robust High-Quality Scheduling of Conditional Plans with Uncertain Durations. *Journal of Artificial Intelligence Research* 56, 123–155 (2016). <https://wufeng02.github.io/doc/pdf/RHWjair16.pdf>. Accessed 2026-02-10.
28. Author(s): Title of the Paper. arXiv preprint arXiv:2510.16034 (2025). <https://arxiv.org/pdf/2510.16034>. Accessed 2026-02-2.
29. PAS: Understanding ISA-18.2 - Management of Alarm Systems for the Process Industries. White Paper, ISA. <https://www.isa.org/getmedia/55b4210e-6cb2-4de4-89f8-2b5b6b46d954/PAS-Understanding-ISA-18-2.pdf>. Accessed 2026-02-10.
30. Nasby, G.: Introduction to Alarm Management and the ISA-18.2 Standard. York Region Presentation (2013). [https://www.grahamnashby.com/files\\_publications/NashbyG\\_2013\\_IntroToAlarmMgmt\\_YorkRegion\\_oct2013\\_slides.pdf](https://www.grahamnashby.com/files_publications/NashbyG_2013_IntroToAlarmMgmt_YorkRegion_oct2013_slides.pdf). Accessed 2026-02-1.